

VS Code Extension Integration Roadmap

Welcome to the IDE-Backend Integration Guide. This document is prepared to help (IDE & Integration Engineer) seamlessly connect the TypeScript VS Code Extension to the AURIX Backend API.

AURIX utilizes a Decoupled Microservice Architecture. As the IDE Integration Engineer, you will operate entirely on the client's local machine and interact with the backend API via standard HTTP requests, without needing to manage AI workers or underlying databases.

Phase 1: Authentication (SecretStorage)

Every request to the backend must be authenticated via Supabase Auth and JWT tokens.

1. **User Login:** Implement an `Aurix: Login` command that prompts the user for credentials and retrieves an `access_token` (JWT) from the backend.
2. **Secure Storage:** Save this token using the native VS Code `SecretStorage` API to maintain persistent sessions.
3. **Request Headers:** Attach the token to every API call using the following format:

```
{ "Authorization": "Bearer <YOUR_JWT_TOKEN>" }
```

Phase 2: Zip & Ship Ingestion

Once the "Pre-Flight Secret Guard" verifies the workspace, the extension must package and stream the local code to the cloud.

- **Endpoint:** `POST /api/scans/upload`
- **Content-Type:** `multipart/form-data`
- **Form Data:**
 - `source_code`: The `.zip` file buffer.
 - `project_id`: The project UUID string.

The backend will stream the zip to Supabase Object Storage and return a `202 Accepted` response. You must save the returned `scan_id` to initiate the polling loop.

Phase 3: The Polling Mechanism

AI analysis and automated exploit/patch generation typically take 2-3 minutes. Use `vscode.window.withProgress` to display a progress notification during this time.

- **Endpoint:** `GET /api/scans/<scan_id>`
- **Task:** Implement an async `setInterval` loop to hit the endpoint every **5 seconds**.
- **Termination:** Stop polling when the status changes from `PENDING` or `SCANNING` to `COMPLETED`.

The final payload will contain a `findings` array with details including `rule_id`, `severity`, `ai_reasoning`, and `patch_code`.

Phase 4: IDE Rendering & AI Upgrades

Upon completion, inject findings directly into the development environment:

1. **Diagnostics UI:** Use `vscode.languages.createDiagnosticCollection` to underline vulnerabilities at the specified `file_path` and `line_number`.
2. **Attack Path Webview:** Build a rich HTML Webview Panel to display the step-by-step exploit chain discovered by the AI Red Team.
3. **"Ghost-Text" Inline Patching:** Utilize the VS Code **Inline Completion API** to stream `patch_code` directly into the editor as grey text, allowing users to accept fixes via the `Tab` key.

Critical Constraints & Error Handling

- **The Node_Modules Nuke:** Strictly enforce `.gitignore` rules during zipping. Uploads exceeding the 10MB limit will be rejected by the backend.
- **Preserve Paths:** Internal folder structures must be preserved in the zip file so the AI Engine can correctly identify and patch files.
- **Backend Failures:** If the API returns `status: "FAILED"`, terminate the progress UI and notify the user via `vscode.window.showErrorMessage`.